Multimodal Cross-Platform Virality Prediction: Using Audio and Metadata Signals Across Spotify and YouTube

Jeslyn Wang, Vancence Ho, Douglas Chun-hao Tan, Muhammad Ammar Bin Mohamad Sofian
1007067, 1007239, 1006656, 1007198
Singapore University of Technology and Design
{jeslyn_wang, vancence_ho, douglas_tan, 1007198}@mymail.sutd.edu.sg

Abstract

This project investigates whether a song's virality on Spotify can predict its virality on YouTube, using a multimodal feature set drawn from audio characteristics and engagement metadata. A merged cross-platform dataset is constructed by linking Spotify track-level audio features with YouTube audio features and engagement metrics for the same songs. Three baseline models are evaluated on Spotify-only data, and a preliminary cross-platform transfer experiment is framed as a binary classification task.

1 Introduction and Motivation

Music virality has ceased to be a single-platform phenomenon. Research tracking over 8,000 tracks across four European countries found that songs achieving higher video streaming volumes on YouTube also achieve higher audio streaming volumes on platforms like Spotify, and crucially, YouTube views tend to lead streams over time, rising and falling before Spotify activity follows suit (RBB Economics, 2017). This cross-platform relationship is further supported by predictive modelling evidence: the incorporation of social media variables from YouTube significantly increased model performance relative to the use of audio features alone, with margins of improvement ranging from 10% to 60%, demonstrating that YouTube engagement signals carry meaningful predictive power for Spotify chart outcomes (Yee and Raheem, 2022). While previous studies have only looked at hit song predictions using either audio features or social media features independently, combining both audio features and social media data offers a more holistic perspective for predicting potential hits. Despite this, most existing virality prediction work treats each platform in isolation, a gap that limits the accuracy and practical utility of such models for the music industry.

The main objective was to study cross-platform music virality by testing whether features from Spotify and YouTube can jointly predict whether a song becomes viral. More specifically, the work framed virality prediction as a supervised binary classification problem and examined whether multimodal signals outperform single-source baselines.

Three related goals were addressed in the project. First, Spotify-style audio and metadata signals were used to build baseline virality models. Second, YouTube-linked audio and engagement-derived features were used to establish a stronger platform-specific baseline. Third, all selected features were combined into a cross-platform model to predict YouTube virality without directly using leakage-prone engagement targets as training features.

2 Literature Review

Virality on audio streaming platforms. Predicting song popularity from Spotify audio features alone has proven to be a limited approach. Yee and Raheem (2022) found that machine learning models trained exclusively on Spotify audio features such as energy, danceability, valence, and tempo performed poorly in classifying songs as hits, suggesting that intrinsic audio characteristics carry insufficient discriminating power on their own.

Virality on video platforms. YouTube engagement signals have demonstrated stronger predictive value. Vallet et al. (2015) showed that YouTube features capturing user engagement, such as views, likes, and ratings, are the most important predictors of video popularity, and that these features alone can achieve high classification accuracy (AUC up to 0.95). Critically, they also demonstrated that cross-system predictions between YouTube and Twitter are feasible, meaning signals from one platform carry meaningful predictive information about behaviour on another.

Cross-platform dynamics. The evidence points consistently toward a promotional relationship between YouTube and Spotify. RBB Economics (2017) found that YouTube views tend to lead Spotify streams over time, rising and falling before audio streaming activity follows, and that tracks with higher initial YouTube exposure go on to achieve greater streams on paid services in subsequent months. Yee and Raheem (2022) reinforced this by showing that adding YouTube social media features to audio-only models improved prediction performance by margins of 10% to 60%, demonstrating that cross-platform signals carry substantial additional information beyond what either platform provides alone.

3 Data Collection

The data collection phase was designed to assemble a comprehensive cross-platform dataset by integrating Spotify track information with YouTube video-level and channel-level data. A systematic retrieval process gathered structured records from multiple digital sources to support downstream analysis. The approach combined automated search, matching, downloading, and metadata extraction, allowing the study to capture both musical attributes and audience engagement signals in a reproducible manner.

3.1 Spotify Dataset

The primary audio dataset is sourced from Kaggle: "Spotify Tracks Dataset". It contains track-level metadata and audio analysis for a large, genre-diverse collection of songs. Each row represents one track and includes features as described in Table 1.

The full list of Spotify features is provided in Table 1 in the Appendix.

3.2 YouTube Dataset

The YouTube dataset was collected to capture engagement-level signals for videos corresponding to songs in the Spotify dataset.

3.2.1 Downloading YouTube Audio Files

To construct the Spotify–YouTube linkage, each Spotify track was queried on YouTube using normalized search strings that removed irrelevant terms such as "official," "lyrics," and "remix." This normalization reduced noise in the retrieval process and improved the likelihood of identifying the correct performance or upload associated with each track. A match confidence score was then computed by comparing track details with YouTube title and channel information, and official sources such as VEVO and label channels were prioritized to improve data quality and reduce mismatches. Once the best candidate was identified, the corresponding audio was downloaded using `yt-dlp`, producing a matched audio corpus for subsequent analysis.

The matching and downloading workflow was engineered for scalability through parallel processing, with eight workers handling the workload concurrently. This design substantially improved throughput and made it feasible to process large numbers of tracks in a reasonable timeframe. To support reliability, the pipeline also implemented resume and checkpoint mechanisms so that progress would not be lost if processing was interrupted. The output of this stage was stored in `combined_match_download_results.csv`.

3.2.2 Collecting YouTube Engagement Metadata

After track matching and audio acquisition, YouTube metadata was collected to enrich the dataset with engagement and channel descriptors. Metadata extraction was performed using `yt-dlp`, retrieving information directly from the video source without consuming API quota. The collected variables included view count, like count, comment count, follower count, verified status, upload date, duration, tags, and uploader information. The resulting dataset was saved as `youtube_metadata.csv`.

In addition to standard engagement metrics, YouTube's viewer retention heatmap data was collected to capture temporal engagement patterns throughout each video. The heatmap represents normalized viewer retention values at regular intervals (approximately every 2.76 seconds), providing insight into drop-off points, replay behavior, and sustained engagement beyond aggregate view counts. Each heatmap is stored as a JSON array of time segments, where `value` represents the normalized proportion of viewers still watching at that time point (1.0 = 100% retention).

3.2.3 Audio Feature Extraction

Following the acquisition of matched YouTube audio files, advanced audio signal processing was performed using the `librosa` library to extract time-series and spectral features from each

track. The feature extraction process utilized a `ThreadPoolExecutor`-based parallel processing architecture with batch processing and checkpoint mechanisms every 500 files to ensure fault tolerance.

Features extracted include MFCCs (20 coefficients capturing timbral texture and spectral envelope characteristics), chroma features (12-bin chromagram representing pitch class distribution), spectral features (spectral centroid, bandwidth, contrast, rolloff, and flatness), temporal features (zero-crossing rate, RMS energy, tempo estimation), harmonic features (Tonnetz representation), and rhythm features (onset strength envelope for beat tracking). For each feature, both mean and standard deviation statistics were computed across time frames, resulting in approximately 80 librosa-derived features per track.

3.2.4 Musical Context Extraction

Following the acquisition of matched YouTube audio files, additional musical context data was extracted through the use of a word2vec model to convert the audio files into skip-gram representations (Herremans and Chuan, 2017). The pipeline funneled these YouTube audio files into an open source mp3 to MIDI converter, Basic Pitch from Spotify (Bittner et al., 2022), before being sliced and made into word2vec embeddings. These embeddings proved inconclusive rather than negative, as the MIDI converter was unable to transcribe many songs properly and the embeddings contributed no signal beyond the Spotify audio feature baseline.

3.3 Merged Cross-Platform Dataset

The two datasets are merged on a composite key of (*track_name*, *artist*), after normalisation of artist name and title strings (lowercasing, punctuation removal, fuzzy matching for slight variations). The resulting merged dataset contains individual songs present in both datasets, with columns representing the union of Spotify audio features, YouTube engagement metrics, and derived virality labels for both platforms.

4 Data Cleaning and Preprocessing

4.1 Spotify Dataset

Following exploratory analysis, we constructed a modeling-ready feature matrix from Spotify's track-level audio descriptors and metadata, with a binary virality label derived from streaming popularity. We removed a single row with missing artist, album, and track text; standardized text fields; and confirmed no duplicate rows. We defined *viral* = 1 if popularity > 50, else 0. The majority of tracks sit below this cutoff, yielding a minority positive class ($\sim$24% viral vs. $\sim$76% non-viral). Identifiers and high-cardinality text fields were dropped, while duration, time signature, tempo, key, mode, loudness, and the seven Spotify perceptual metrics were retained. Categorical variables (`explicit` and `track_genre`) were encoded using one-hot encoding with `drop_first=True`.

4.2 YouTube Dataset

Engagement metrics including `view_count`, `like_count`, and `comment_count` exhibited strong right-skewness and were log-transformed using $\log(x + 1)$. Channel-level imbalance was addressed through `channel_subscriber_count` bucketing into quintiles. Temporal features were engineered from `publish_date`, decomposed into day-of-week, month, and days since upload to capture recency and seasonal upload patterns.

Engagement-normalised features were constructed to better represent user interaction relative to exposure:

$$\text{like_rate} = \frac{\text{like_count}}{\text{view_count}}$$

$$\text{comment_rate} = \frac{\text{comment_count}}{\text{view_count}}$$

$$\text{total_engagement_rate} = \frac{\text{like_count} + \text{comment_count}}{\text{view_count}}$$

A composite virality score was constructed as a weighted combination of normalised log-transformed view count (weight: 0.60) and normalised total engagement rate (weight: 0.40). Tracks in the top quartile (75th percentile) of the training-set score distribution are labelled viral, classifying 7,939 songs (25.0%) as viral and 23,814 (75.0%) as non-viral.

5 Exploratory Data Analysis

5.1 Spotify EDA

Dataset Characteristics. The Spotify dataset comprised 113,999 tracks after removing 1 duplicate. Popularity scores exhibited a right-skewed distribution (mean = 33.24, median = 35.00, std = 22.31, range = 0–100). Using a popularity > 50 threshold, the dataset classified 27,770 tracks

(24.4%) as viral and 86,229 tracks (75.6%) as non-viral.

Audio Feature Distribution Analysis. Danceability (mean = 0.567, median = 0.580) and energy (mean = 0.641, median = 0.685) showed moderate-to-high values. Instrumentalness exhibited strong right-skew (mean = 0.156, median = 0.000042). Duration averaged 228,029 ms (3.8 minutes), and tempo centered around 122 BPM, consistent with contemporary pop and dance music conventions.

Feature Correlation with Virality. Correlation analysis between audio features and the viral label revealed uniformly weak relationships (all $|r| < 0.10$). The strongest correlations were liveness ($r = -0.087$), instrumentalness ($r = -0.082$), and danceability ($r = +0.052$). These uniformly weak correlations motivated the incorporation of fine-grained librosa-extracted acoustic features and YouTube-derived engagement patterns.

5.2 YouTube EDA

Dataset Characteristics. The merged cross-platform dataset comprised 31,753 songs. YouTube engagement metrics exhibited heavy right-skew (view count: median = 693,430, mean = 45.2M, max = 7.0B), requiring log transformation. Using the composite virality score, the top 25% threshold (0.396) classified 7,939 songs (25.0%) as viral.

Feature Correlation Analysis. Spotify audio features demonstrated moderate correlation with YouTube virality (Spearman $r = 0.310$), with energy, danceability, and valence showing positive associations while instrumentalness exhibited negative correlation.

Heatmap Engagement Pattern Analysis. Analysis of YouTube viewer retention heatmaps (31,753 videos) revealed temporal engagement patterns distinguishing viral content. The strongest heatmap predictor was decay rate ($r = 0.221$), indicating viral videos maintain viewer attention more consistently. First 10-second retention ($r = 0.147$) and quartile drop ($r = 0.147$) also showed predictive value. Counterintuitively, replay-related features (rewatch ratio $r = -0.144$, peak height $r = -0.151$) showed negative correlations, suggesting viral content achieves broad reach through sustained first-time viewing rather than repeated engagement from smaller audiences.

After correlation-based filtering (threshold = 0.01), the final feature set comprised: Spotify features (11), librosa audio features (77 from 89), heatmap engagement features (27 from 30), and YouTube metadata (5), totaling 95 features.

6 Problem Definition and Models

6.1 Problem Statement

Three related prediction tasks are addressed:

1. **Task 1 (Single-Platform Baseline):** Given Spotify audio features for a track, predict whether the track is viral on Spotify ($y_{\text{Spotify}} \in \{0, 1\}$).

2. **Task 2 (Single-Platform Baseline):** Given YouTube audio and metadata features for a track, predict whether the track is viral on YouTube ($y_{\text{YouTube}} \in \{0, 1\}$).

3. **Task 3 (Cross-Platform Transfer):** Given Spotify audio features and YouTube metadata features, predict whether the track will be viral on YouTube ($y_{\text{YouTube}} \in \{0, 1\}$).

Task 3 is the core research question: can signals from one platform's content domain generalise to another's engagement-based virality outcome?

6.2 Spotify Baseline Models (Task 1)

Three models are trained and compared on the Spotify virality prediction task:

Decision Tree (DT). A single classification tree on the engineered Spotify feature matrix. It partitions the feature space with axis-aligned rules, capturing thresholds and interactions in one hierarchy without boosting or bagging. It sets a lower bound for what tree-shaped decisions can do before adding randomisation or sequential boosting.

Gradient Boosted Trees (XGBoost). A strong tree ensemble capable of capturing non-linear feature interactions. XGBoost is the most commonly adopted model in Spotify popularity prediction benchmarks and frequently achieves the best single-platform accuracy.

Random Forest (RF). A bagging ensemble of many deep trees, each trained on a bootstrap sample of rows and a random subset of features at each split. This decorrelates trees and reduces variance compared to a single decision tree, while handling non-linearity and mixed feature types without a linearity assumption.

6.3 YouTube Baseline Model (Task 2)

A LightGBM classifier was trained on the YouTube dataset. The dataset was split into training and testing subsets prior to any scaling or target construction to prevent data leakage. Feature scaling used Min-Max normalisation fitted exclusively on training data. The virality threshold was computed at the 75th percentile of the training-set score distribution and applied consistently to avoid leakage. All variables directly used in the construction of the virality score were excluded from the final feature set.

6.4 Ensemble Methods (Task 3)

A unified dataset with a consistent target definition (YouTube virality) was constructed, enabling meaningful comparison and ensembling. Three tree-based models (Random Forest, XGBoost, LightGBM) were trained, with hyperparameters optimised using `RandomizedSearchCV` with cross-validation. Random Forest was tuned using F1-score to address class imbalance; XGBoost and LightGBM were tuned using ROC-AUC.

A stacking ensemble was implemented with the three base models as Level 0 learners. Out-of-fold predictions were generated using cross-validation to produce meta-features, which were passed to a Logistic Regression meta-learner (Level 1) to produce the final prediction.

7 Evaluation Methodology

7.1 Dataset Splits

All models are evaluated using stratified 5-fold cross-validation to account for class imbalance (approximately 20% positive class). For the cross-platform task, a creator/artist-level split is applied, such that all tracks by a given artist appear exclusively in either the train or test fold.

7.2 Evaluation Metrics

The evaluation metrics used are summarised in Table 2 in the Appendix.

8 Results and Discussion

8.1 Spotify Models Results

8.1.1 Model Performance

Results are reported in Table 3 in the Appendix.

Overall, both ensemble models demonstrate strong predictive capability, with XGBoost outperforming Random Forest in most evaluation metrics. Random Forest achieves an accuracy of 83.3%, while XGBoost achieves a higher accuracy of 85.1%, indicating that XGBoost provides slightly better overall classification performance.

However, when examining precision, Random Forest records a higher value of 0.765, compared to 0.727 for XGBoost. This suggests that Random Forest makes fewer false positive predictions and is more conservative when predicting positive cases. In contrast, XGBoost performs better in terms of recall, achieving 0.608 compared to Random Forest's 0.502, indicating that XGBoost is more effective at identifying actual positive cases and reduces the number of false negatives.

The F1-score further highlights the trade-off between the two models. XGBoost achieves a higher F1-score of 0.662, compared to 0.606 for Random Forest. Since the F1-score balances precision and recall, this result suggests that XGBoost provides a better overall balance between correctly identifying positive cases and maintaining prediction accuracy.

8.1.2 Feature Importance Analysis

We inspected tree-based feature importance for the tuned Random Forest and XGBoost models using each estimator's `feature_importances_` attribute (Gini mean decrease in impurity in scikit-learn). Importance vectors were sorted and plotted as horizontal bar charts with feature names taken from the modelling matrix `X.columns` (127 predictors after one-hot encoding of `explicit` and `track_genre`, plus continuous Spotify audio descriptors).

Both ensembles allocate substantial importance across (i) core audio descriptors—typically loudness, energy, danceability, acousticness, valence, tempo, and duration, among others—and (ii) a subset of `track_genre_*` indicator columns. That pattern is consistent with the EDA: virality rates differ markedly by genre, so genre membership is a strong context for how audio features relate to the label. At the same time, the correlation heatmap of audio features with continuous popularity shows modest pairwise linear associations; tree models can still exploit thresholds and interactions (e.g., energy conditional on genre) that do not show up as large single correlations.

Rankings are qualitatively similar but not identical between RF and XGBoost: bagging (RF) spreads importance across many trees and feature subsamples, while boosting (XGBoost) can concentrate importance on features that most reduce loss along the boosting path. We do not treat im-

portances as causal: MDI can be biased toward high-cardinality features (many genre dummies), and a high rank does not prove a feature drives virality outside the Spotify catalogue.

8.1.3 Model Behavior and Generalisation

Models were trained on a single hold-out split: 80% train / 20% test, `random_state=42`, using the full engineered matrix from `spotify_tracks_cleaned.csv` (no stratification on `viral` in the split call). Hyperparameters for RF and XGBoost were tuned via search and cross-validation on the training portion, and final metrics were reported on the held-out test set.

Train vs. Test Behaviour. For the tuned Random Forest, notebook outputs show a material gap between training accuracy ($\sim$0.90) and test accuracy ($\sim$0.82), which is typical of flexible tree ensembles when not fully variance-regularised. For tuned XGBoost, training and test accuracies are closer ($\sim$0.82 vs. $\sim$0.80), suggesting a somewhat better bias–variance trade-off on this split, consistent with XGBoost's higher test F1 in Table 3.

Class Imbalance & Metric Choice. EDA fixes the positive rate near 24% viral vs. $\sim$76% non-viral. Accuracy is therefore dominated by the majority class. The precision–recall pattern (RF more precise, XGBoost higher recall) reflects different operating points on the same imbalance: a conservative positive predictor (fewer false "viral" labels) vs. one that casts a wider net for true virals (fewer false negatives). F1 summarises that trade-off; reporting ROC-AUC or PR-AUC in addition would further stress ranking quality under imbalance.

Scope of Generalisation. Results generalise to i.i.d. draws from the same Spotify tracks table and label rule (`popularity` > 50). They do not by themselves guarantee transfer to other markets, time periods, or platforms. Leakage was controlled in preprocessing by dropping popularity from predictors; remaining risk is catalog bias (genre and audio feature distributions tied to this dataset).

8.2 YouTube Models Results

8.2.1 Model Performance

Results are reported in Table 4 in the Appendix.

LightGBM demonstrates superior performance across all evaluation metrics compared to the Decision Tree model. Specifically, LightGBM achieves an accuracy of 81.6%, which is significantly higher than the Decision Tree's accuracy of 73.8%. In terms of precision, LightGBM records a value of 0.648, outperforming the Decision Tree's precision of 0.482, suggesting that LightGBM produces fewer false positive predictions and is more reliable when predicting positive instances.

A more notable difference is observed in recall, where LightGBM achieves 0.606 compared to 0.402 for the Decision Tree, indicating that LightGBM is considerably more effective at identifying actual positive cases. The F1-score further confirms this trend, with LightGBM achieving 0.626 while the Decision Tree attains only 0.438.

8.2.2 Feature Importance Analysis

The feature importance analysis reveals the most influential variables contributing to the LightGBM model's predictions. Out of a total of 77 features, the top 10 most important features are primarily derived from Mel-Frequency Cepstral Coefficients (MFCC), spectral contrast, and root mean square (RMS) energy.

The most significant feature is `mfcc_1_std`, followed closely by `spectral_contrast_7_std` and `rms_std`. Several other MFCC-based features, such as `mfcc_7_mean`, `mfcc_10_mean`, `mfcc_9_mean`, and `mfcc_4_mean`, also rank highly. Additionally, both mean and standard deviation values of spectral contrast (e.g., `spectral_contrast_1_mean`, `spectral_contrast_7_mean`) and RMS (e.g., `rms_mean`) appear among the top contributors.

This distribution indicates that the model relies heavily on both frequency-domain features (MFCC and spectral contrast) and energy-related features (RMS). The presence of both mean and standard deviation statistics suggests that not only the average characteristics of the audio signals are important, but also their variability over time. Furthermore, the importance is relatively spread across multiple features rather than being dominated by a single variable, implying that the classification task depends on a combination of acoustic properties.

8.2.3 Model Behavior and Generalization

The LightGBM model's reliance on a diverse set of features, including MFCC, spectral contrast, and RMS, indicates that it is not over-dependent on a narrow subset of variables. Instead, it captures a wide range of acoustic characteristics, including timbral texture, frequency distribution, and signal energy. This diversity reduces the risk of overfit-

ting and enhances the model's ability to generalize to unseen data. The inclusion of both mean and standard deviation features enables LightGBM to model more complex patterns compared to simpler algorithms such as Decision Trees.

8.3 Combined Spotify–YouTube Virality Prediction

8.3.1 Model Performance

To evaluate the contribution of cross-platform features, three baseline models (Random Forest, XGBoost, LightGBM) and a stacking ensemble were trained on the combined multimodal feature set. This dataset integrates Spotify audio descriptors (11 features), librosa-extracted acoustic features (77 features), and heatmap-derived temporal engagement patterns (27 features), totaling 95 features after exclusion of direct engagement metrics (`view_count`, `like_count`, `comment_count`, `like_rate`, `comment_rate`).

Results are reported in Table 5 in the Appendix.

All models achieved 5-fold cross-validation scores between 0.822 and 0.836, demonstrating robust generalization. The stacking ensemble, which combines predictions from all three base models via a logistic regression meta-learner, achieved the best overall performance with 85.1% test accuracy and 0.853 AUC.

8.3.2 Feature Importance Analysis

Feature importance analysis from the Random Forest model revealed the relative contribution of each feature modality. Among heatmap features (27 features), `decay_rate` had the highest overall importance, `first_10s_retention` was a strong predictor, and `quartile_drop` showed moderate importance. Among librosa audio features (77 features), spectral characteristics (`spectral_centroid`, `spectral_bandwidth`), MFCC coefficients capturing timbral texture, and temporal features (zero-crossing rate, RMS energy) were most prominent. Among Spotify features (11 features), energy, danceability, and valence showed moderate positive correlations with virality, while instrumentalness showed negative correlation.

The prominence of heatmap features demonstrates that temporal engagement patterns provide complementary predictive signals beyond traditional audio representations.

8.3.3 Model Behavior and Generalization

The results highlight the difference between accuracy and true predictive performance. Although all models achieve 66–80% accuracy, this is misleading due to class imbalance (≈75% non-viral), where a naive "always non-viral" model already reaches 75% accuracy. Therefore, F1 score is a more appropriate metric.

Across models, F1 scores remain low (0.29–0.42), showing difficulty in identifying viral songs. XGBoost performs worst (F1 = 0.29), indicating poor recall for the viral class. LightGBM shows strong overfitting, with AUC dropping from 0.97 (train) to 0.79 (test), while Random Forest also shows a notable generalisation gap. The Stacking Ensemble performs best overall due to its more consistent train-test performance, achieving 80.3% accuracy, 0.790 AUC, and 0.424 F1. Its strength comes from combining multiple models, reducing variance and overfitting. Overall, consistently low F1 scores indicate a class imbalance problem where models are biased toward non-viral tracks. Techniques like class weighting or SMOTE are recommended to improve viral-class detection.

8.3.4 Cross-Platform Prediction Insights

Achieving 82–85% accuracy without access to view counts demonstrates that virality signals are embedded in audio characteristics and temporal viewing behavior. The importance of heatmap features (particularly `decay_rate` and early retention) indicates that how audiences engage with content over time is as predictive as acoustic characteristics.

The negative correlation of replay-related features with virality is counterintuitive: broadly viral content achieves reach through sustained first-time viewing rather than repeated engagement from smaller audiences. This distinguishes mass-market viral content from niche content with devoted followings.

9 UI Implementation

A Streamlit app was built in Python to demonstrate the ensemble model, allowing the user to input a series of embeddings of a processed song. The app first loads the ensemble model from Hugging Face, before running these embeddings through the model and outputting a virality score and a statement on the song's potential to go viral.

Acknowledgments
This project proposal was developed with reference to the document *project_description.pdf*, which was provided as part of the course materials. The structure and task descriptions outlined in the document served as the foundations for this work. Additionally, this proposal was prepared and written using the official ACL LaTeXtemplate distributed by the *Association for Computational Linguistics*.

Code Availability
The source code supporting this report is available at: https://github.com/vancenceho/viral-content-predictor

Appendix

A Combined Ensemble Pipeline

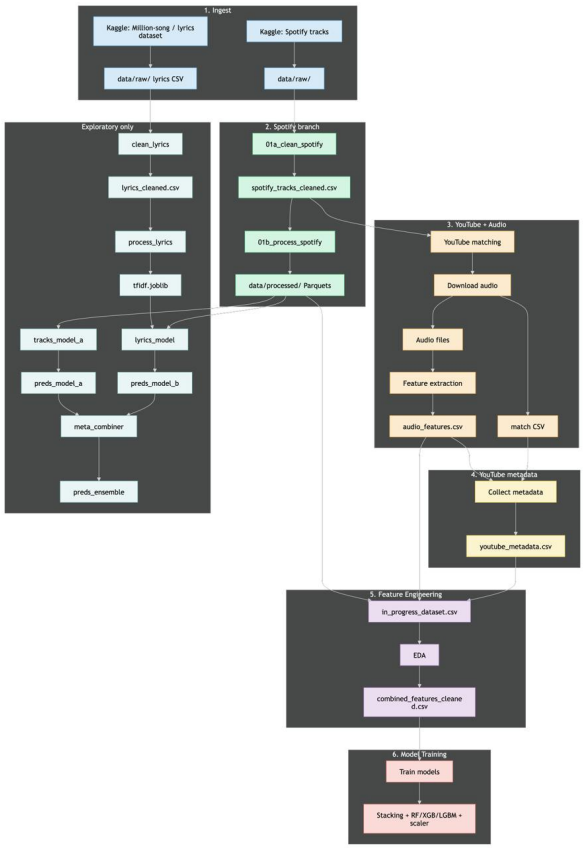

Figure 1: Combined Ensemble Pipeline

The full pipeline is organized into six stages: (1) data ingestion from Kaggle datasets, (2) Spotify feature processing, (3) YouTube matching, audio downloading, and feature extraction, (4) YouTube metadata collection, (5) feature engineering and EDA, and (6) model training with stacking ensemble construction (RF + XGB + LGBM + meta-learner scaler).

B Tables

Feature	Description
track_id	Spotify unique identifier
artists	Performing artist(s)
track_name	Title of track
popularity	Spotify popularity score
duration_ms	Track duration in milliseconds
explicit	Explicit content flag
danceability	Rhythmic regularity suited for dancing
energy	Perceptual intensity and activity
key	Musical key (pitch class notation)
loudness	Overall loudness
mode	Major (1) or minor (0) modality
speechiness	Presence of spoken words
acousticness	Confidence the track is acoustic
instrumentalness	Likelihood of no vocals
liveness	Probability of live performance
valence	Musical positivity/happiness
tempo	Estimated tempo in BPM
time_signature	Estimated beats per bar
track_genre	Genre label

Table 1: Spotify Track Dataset Features

Metric	Rationale
AUC-ROC	Primary metric; threshold-independent measure of discriminative ability, robust to class imbalance
F1-Score (macro)	Harmonic mean of precision and recall, penalising models that ignore the minority class
Accuracy	Reported for completeness; interpreted alongside AUC given class imbalance
Average Precision (AP)	Area under the precision-recall curve; more sensitive to minority class performance than AUC
Calibration (Brier Score)	Measures whether predicted probabilities are well-calibrated, important if outputs are used for ranking

Table 2: Model evaluation metrics and their rationale

Model	Accuracy	Precision	Recall	F1
Decision Tree	77.3%	0.528	0.523	0.525
Random Forest	83.3%	0.765	0.502	0.606
XGBoost	85.1%	0.727	0.608	0.662

Table 3: Spotify model performance

Model	Accuracy	Precision	Recall	F1
LightGBM	81.6%	0.648	0.606	0.626
Decision Tree	73.8%	0.482	0.402	0.438

Table 4: YouTube model performance

Model	Accuracy	AUC	Precision	Recall	F1
Random Forest	66.2%	0.755	0.375	0.726	0.494
XGBoost	79.0%	0.768	0.636	0.184	0.285
LightGBM	80.0%	0.792	0.667	0.240	0.353
Stacking Ensemble	80.3%	0.790	0.633	0.319	0.424

Table 5: Combined model performance on test set

C UI Screenshots

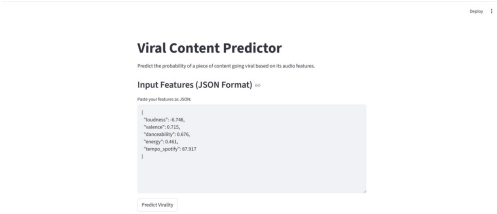

Figure 2: Streamlit UI before features are input

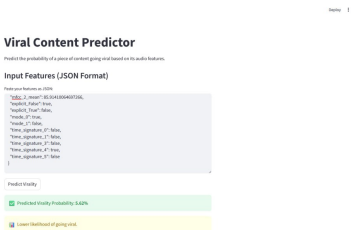

Figure 3: Streamlit UI after features have been input, showing predicted virality score

References

Rachel M. Bittner, Juan José Bosch, David Rubinstein, Gabriel Meseguer-Brocal, and Sebastian Ewert. 2022. A lightweight instrument-agnostic model for polyphonic note transcription and multipitch estimation. In *ICASSP 2022 – 2022 IEEE International Conference on Acoustics, Speech and Signal Processing*.

Dorien Herremans and Ching-Hua Chuan. 2017. Modeling musical context with word2vec. *Proceedings of the First International Workshop on Deep Learning and Music joint with IJCNN*, 1(1):11–18.

RBB Economics. 2017. Value of youtube to the music industry – paper IV: Value for consumers. Technical report, RBB Economics.

David Vallet, Shlomo Berkovsky, Sebastien Ardon, Anirban Mahanti, and Mohamed Ali Kaafar. 2015. Characterizing and predicting viral-and-popular video content. In *Proceedings of the 24th ACM International Conference on Information and Knowledge Management (CIKM '15)*, pages 1591–1600.

Yong Kai Yee and Mahwash Raheem. 2022. Predicting music popularity using Spotify and YouTube features. *Indian Journal of Science and Technology*, 15(36):1786–1799.